

Borsod Rally

Szobek Ádám Márk
Konzulens: Somogyi Ferenc Attila

A Borsod Rally egy versenyzős játék, melyet hallgatótársammal, Kovács Nimród Ányossal készítettünk a 2024/25-ös tanév tavaszi félévében induló Önálló labor című tárgy keretében.

A játék célkitűzései a következők voltak:

- Procedurális pályagenerálás
- Gépi tanulással betanított ellenfelek
- Többjátékos mód

A fent felsorolt pontok közül az én feladatom volt az ellenfelek betanítása és a többjátékos mód megvalósítása, míg Nimród a pályagenerálással foglalkozott.

Gépi tanulással betanított ellenfelek

Ez a része a projektnek arra fókuszált, hogy vajon be tudunk-e tanítani egy gépi tanulás alapú modellt, hogy az képes legyen egy véletlenszerűen generált pályán is jól teljesíteni.

Természetesen még a legtapasztaltabb versenyzők is rosszabbul teljesítenek egy számukra ismeretlen pályán (értelemszerűen nehéz teljes bátorsággal belemenni egy olyan kanyarba, melynek sem az élességét, sem a hosszát nem ismerjük), melyet a projektmunka előtti kutatás is igazolt (erről később bővebben lesz szó).

Mielőtt belevágtam volna a feladatba, természetesen feltúrtam az internetet, hátha valaki csinált már hasonló dolgot. A problémánkhoz legjobban hasonlító próbálkozás talán [itt](#) található, ahol a posztoló egy olyan modellt tréningezett, amely képes a Trackmania nevű játékon belül vezetni. Ez a modell szépen teljesített, azonban mivel a modell tisztán vizuális adatokat kap (leszámítva egy-két, az autó állapotát leíró számot), a fentebb említett problémánk, a pályaismeret hiánya itt is korlátozó tényező.

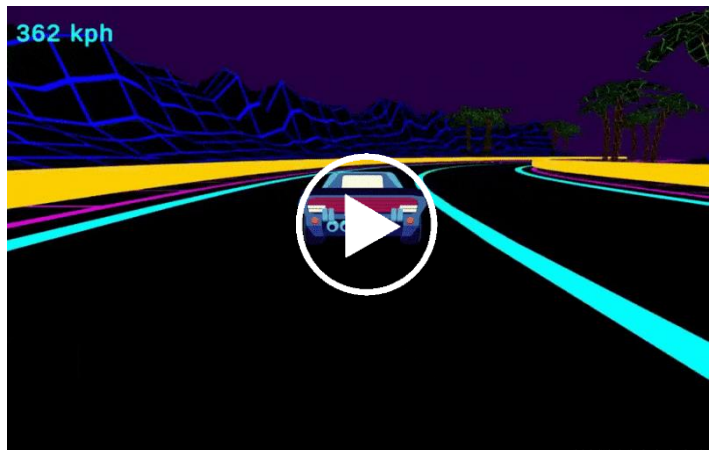
Az erre a problémára adott megoldás jellemzően az volt, hogy az ágenset az összes pályára külön betanítják, amiről az egyik legjobb forrás, amelyet találtam az egy, [az EA Codemasters egyik tagja által adott előadás](#) volt. Az előadás által említett játékban nem csak pályánként voltak különböző modellek betanítva, hanem pálya-autó páronként.

Összefoglalva, a feladatomban az, hogy a modellt olyan adatokkal lássam el, amelyekből nem csak az aktuális helyzetet tudja felmérni, hanem (valamilyen szinten) ismerje a pálya későbbi elrendezését is.

A modell tanítására a legkézenfekvőbb választás a Unity beépített **MLAgents** nevű csomagja. Az MLAGents egy pytorch-ra épült csomag és az a célja, hogy egyszerűbbé tegye a modell tréningezést a Unity fejlesztő környezetén belülről, így ez teljesen megfelel a céljainknak.

Ezek után az első tréning környezet elkészítése következett. Ez egy kicsit trükkös feladat volt, mivel ez még a projekt nagyon korai szakaszában történt, szóval Nimród még távol volt attól, hogy egy véletlenszerűen generált pályát tudjon adni nekem használatra (ami pedig egy elengedhetetlen része a tanításnak), így átmenetileg egy korábbi projektemből¹ emeltem át a pályát (ez egy jóval egyszerűbb megoldás a pályagenerálásra, mint amit majd Nimród fog csinálni).

Mivel ezelőtt még nem használtam gépi tanulást ilyen célra, így hogy egy kicsit megismerkedjek mind magával a tanítással, mind az MLAGents csomag által adott funkciókkal (és megkötésekkel, erről majd később), egy olyan környezetet készítettem, ahol az ágensnek végtelen tapadása van (szóval a kanyar sugara független az autó sebességétől). Bár nagyon szórakoztató volt, ahogy az autó egy vadászpilót megszégyenítő sebességekkel veszi be a kanyarokat (a rekord 600+ km/h volt, mely a Föld felszíni légkörében kb. Mach 0,5), ez sajnos nem sokkal vitt közelebb a probléma megoldásához, mivel ha az autó sosem csúszik meg, akkor az sem szükséges, hogy a sofőr „megtervezze”, hogy hogyan veszi be a kanyart.



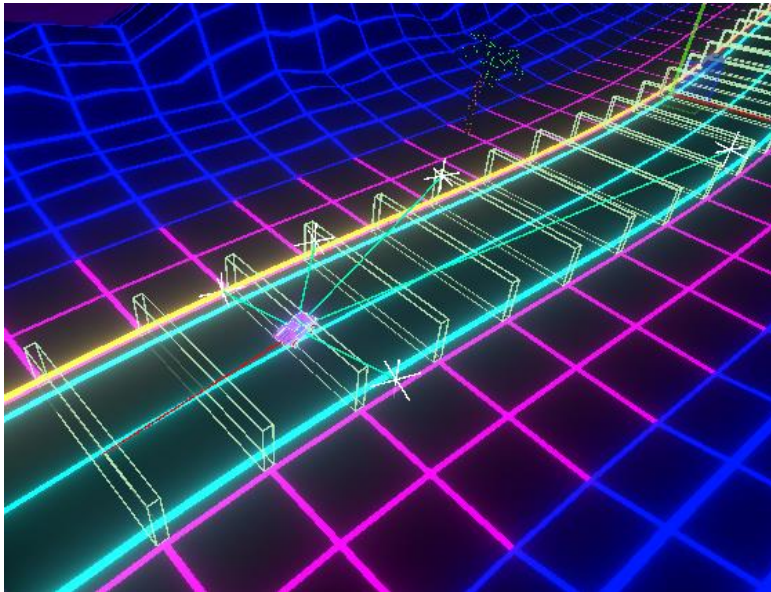
Megjegyzés: A következő szakaszhoz tartozó felvételeim sajnos elvesztek, mert – egy jó informatikushoz illő módon – elfelejtettem biztonsági mentést készíteni róluk mielőtt újratelepítettem a Windows-t a gépemen.

Az első modell, amelyet rendes fizikai szimulációval tréningeztem, a következő módon működött:

- Amit megkap
 - A távolság a falaktól bizonyos irányokba
 - A sebességvektor és az előre irány által bezárt szög (mennyire driftel)
 - A pálya iránya az adott ponton

¹ Hogy egy kis reklámot is beszúrjak, az említett „korábbi projekt” az Initial E nevű játék, mely megtalálható [Google Play-en](https://play.google.com/store/apps/details?id=com.initial.e) ;)

- Cselekvések
 - Folytonos kimenet a kormányzásra $[-1;1]$ intervallumon, -1 teljesen balra, 1 teljesen jobbra
 - Folytonos kimenet az előre-/hátramenetre (-1 teljesen hátra, 1 teljesen előre)
- Jutalmak
 - Sebesség jutalmazása (nem lineárisan)
 - Egyhelyben állás (súlyos) büntetése
 - Haladás jutalmazása egy ellenőrzőpont-rendszerrel
 - Fal érintése esetén új tanulási epizód kezdése



A képen az első „rendes” tanuló környezet látható. A kék vonalak a kis keresztekkel a végüknél a faltól való távolságot jelzik, a kis dobozocskák pedig az ellenőrzőpontok.

Az előző modellnek két fő problémája volt:

- A modell szeretett néha egy helyben megállni és utána nem mozdulni (annak ellenére, hogy ez a sebesség jutalmazásánál implicit, az álldogálás büntetésénél pedig explicit büntetve volt)
- A modell sosem tanulta meg, hogy kell tolatni

Megjegyzés: Az ezt követő körülbelül egy hét azzal ment el, hogy a modellem hirtelen jóval rosszabbul teljesített, mint korábban. Elég sok mindent megpróbáltam az ez ellen való küzdelem során, de mint utólag kiderült, csak lemaradt egy nulla a jobb oldalt látható konfigurációs paraméterről, mely teljesen haszontalanná tette az agyat.

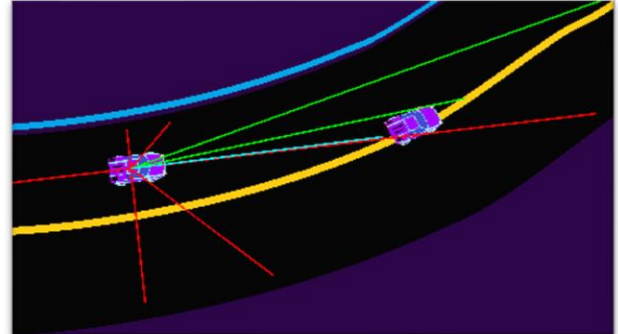
```
epsilon_schedule: 1.0e-4
network_settings:
  normalize: false
  hidden_units: 140
  num_layers: 6
```

Az egyhelyben megállós problémát egy leszámítolós rendszer bevezetésével kezeltem, mely az ellenőrzőpontok elérését egy konstans jutalom helyett annak függvényében pontozza, hogy mennyi ideig tartott elérni az adott ellenőrzőpontot.

A tolatásképtelenséget pedig úgy sikerült kikselezni, hogy a modell az eddigi folytonos kimenet helyett egy diszkrét (korábban 5, végül csak 2 fokozatú) kimenettel tudja befolyásolni, hogy előre vagy hátra szeretne menni.

Az előző ötleteket felhasználva a végső modell a következőképpen működik:

- Bemenetek
 - Falaktól való távolság (piros vonalak)
 - Van-e kocsi előtte (türkiz vonal)
 - Pálya iránya (zöld vonala)
 - Sebességvektor és az előre irány által bezárt szög
 - Sebesség nagysága
- Kimenetek
 - Folytonos kimenet a kormányzásnak
 - Diszkrét (2 fokozatú) kimenet az előre-/hátramenetnek
- Jutalmak/büntetések
 - Az autó falat érintve „meghal”
 - A jutalmak szakaszfüggő (bővebben lejjebb)



A környezet ekkor már nem az Initial E-ből átemelt pályát használta, hanem a Nimródféle pályagenerálás többé-kevésbé kész változatát.

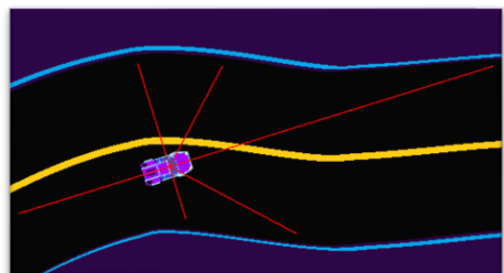
A tanulás meggyorsítása végett párhuzamosan több ágenszt szimulálok (az egyéni szakaszoknál 5, a közösnel pedig 4). Ezekről jobboldalt látható illusztráció.



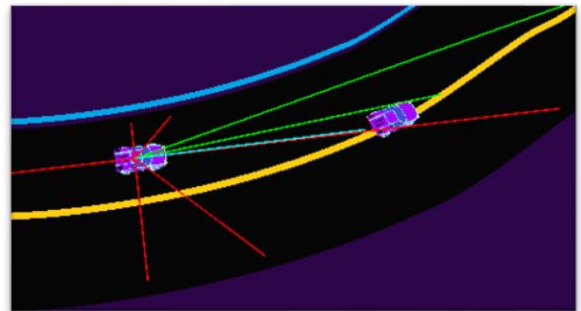
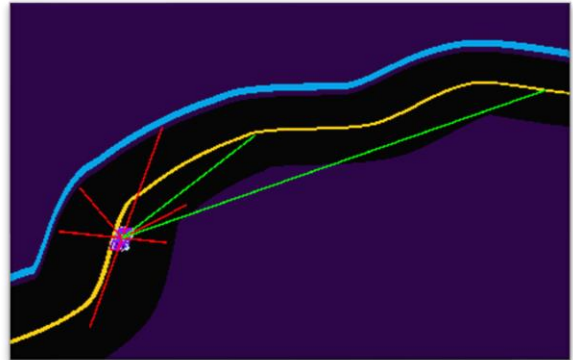
A korábbi bekezdésekben többször is megemlítettem bizonyos *szakaszokat*. Na de mik ezek? Hogy az ágenset ne egyből a mélyvízbe dobjuk, csak fokozatosan kapja meg a bemeneteket. A szakaszok a következőképpen néznek ki:



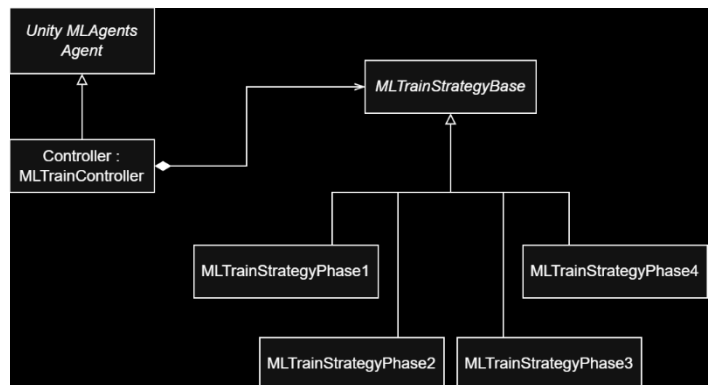
- 1. szakasz
 - Ez a szakasz arra van, hogy az ágensnek kialakuljon egy elképzelése arról, hogy mi az a gáz és kormányzás
 - Az ágens ekkor csak a falaktól való távolságot kapja meg bemenetként.



- A leszámítolós rendszer itt még nem aktív, csupán a sebesség van jutalmazva
- Tanulási idő 10-20 perc
- 2. és 3. szakasz
 - Itt a modell megtanulja, hogyan kell gyorsan végigmenni a pályán
 - A második szakaszban az ágens megkapja a közelebbi pályapont irányát (rövidebb zöld vonal) és a sebesség nagyságát
 - A harmadik szakaszban a távolabbi pályapont iránya (hosszabb zöld vonal) és a sebesség és az előre irány által bezárt szög is ismert
 - A leszámítolós rendszer itt már aktív
 - Tanulási idő: 40-60 perc
- 4. szakasz
 - Az ágens a külön pálya helyett most egy pályán versenyeznek
 - Az utolsó bemenet, az előttünk levő autó távolsága is ismert
 - Tanulási idő: 40-60 perc



Mivel nagyon sokszor kellett újratanítani a modelleket, igyekeztem minél kényelmesebbé tenni a folyamatot, így ahelyett, hogy szakaszonként különböző ágenszt írnék, inkább kiszerveztem a jutalmazást és az információgyűjtést a Strategy mintának megfelelően, mely a mellékelt diagramon látható. Ez annyiban megkönnyítette a betanítást, hogy szakaszváltásnál a kódba való írogatás helyett a Unity Editorból is ki tudtam cserélni az éppen aktuális szakaszt.



Bővebben a 4. szakasz megvalósításáról:

Az általam választott tanítási mód a közös szakaszra a következőképpen néz ki; az ágens elindulnak a rajtvonalról, és addig tart az epizód, ameddig van olyan ágens, amely nem végzett (nem ment falnak és nem érte el a célvonalat), ami azt jelenti, hogy a többi, már készen levő ágens eddig várakozik.

A Unity MLAgents csomag úgy van megtervezve, hogy ha az ágens az epizód végére ért, akkor egyből újakezdi azt, mely a mi céljainknak most nem megfelelő.

Na de hogyan tudom várakoztatni az ágenseket? Erre az első megoldás az volt, hogy deaktiválok minden kimenetet és bemenetnek alapértékeket adok meg. Ez természetesen csak átmeneti megoldás lehetett, hiszen ilyenkor nem befejeztem a tanítást, hanem csak haszontalan értékekkel bombázom az ágenset.

A végső megoldás (hosszas MLAgents dokumentáció olvasgatás után) az lett, hogy deaktiválom az ágenshez csatolt DecisionRequester komponenst, így teljesen megakasztva a tanítást.

A fentieket bevetve egy egész kompetens sofőrt sikerült betréningezni, azonban ez még mindig csak egy közepesen jó játékos szintjét éri el sajnos.

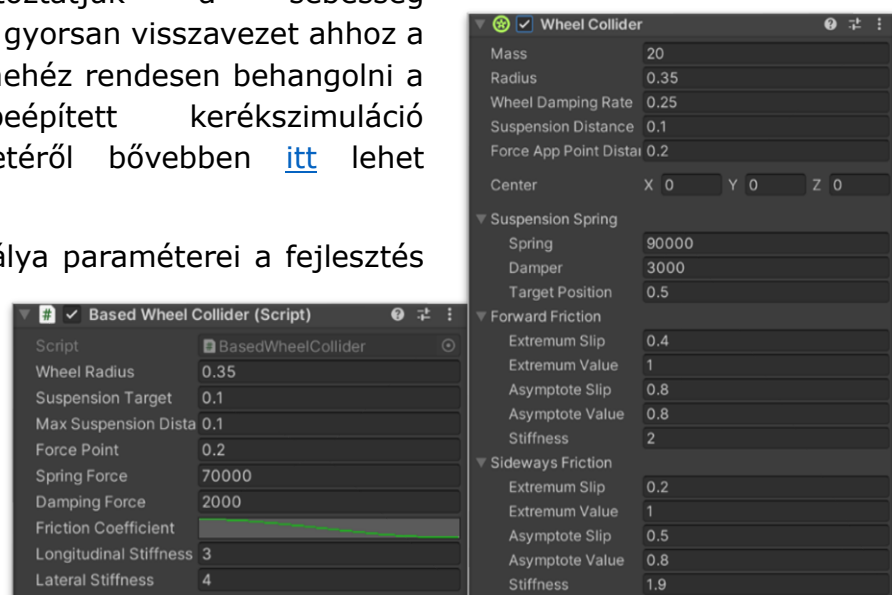
A korai tervekben különböző erősségű ellenfeleket is be akartam tanítani, azonban erre sajnos nem jutott idő, bár egy érdekes továbbvitele lenne a projektnek.

Továbbá az általánosítás sem volt teljes siker, mert a tapasztalat azt mutatja, hogy ha az ágens nagyon kanyargós pályán lett betanítva, akkor az ha esetleg egy egyenesebb pályára kerül, elég félénken bánik a gázpedállal. Ennek kezelésére 3 különböző modellt tréningeztem, mindet más kanyargóssági szinttel.

Saját kerékszimuláció

A projekt korai szakaszában sok időm elment arra, hogy megpróbáljam behangolni a Unity beépített WheelCollider-ét, sajnos sikertelenül. A probléma, amelybe mindig belefutottam, hogy az a kerék konfiguráció, amely működik alacsony sebességnél, az magasnál nem (és vice versa). Továbbá ami minket érzékenyen érintett, az az, hogy a Unity WheelCollider-e a leszorítóerőt csak azután veszi be a képbe, miután már eldöntötte, hogy az autó csúszik-e vagy sem (erre egy nagyon csúnya megoldás az, hogy a leszorító erő kifejtése helyett a kerék súrlódási paramétereit változtatjuk a sebesség függvényében, de ez gyorsan visszavezet ahhoz a problémához, hogy nehéz rendesen behangolni a kereket). A beépített kerékszimuláció hangolásának menetéről bővebben [itt](#) lehet olvasni.

Mivel a kocsí és a pálya paraméterei a fejlesztés alatt gyakran változhatnak, nem akartunk minden egyes alkalommal órákat eltölteni a kerék újrahangolásával, ezért a Unity-s



WheelCollider és az [ebben a videóban](#) elkészített kerék összekombinálásával csináltam egy saját kerék szimulációt.

Az eredmény egy jóval egyszerűbben beállítható kerék lett, mely a paraméterek számán is látszik.

Az új kerék egy egyszerűsített modellt használ (a keréknek nincs tömege, a felfüggesztésnek nincs minimális hossza), mely egy elég „árkádós” érzetet ad a kocsinak. Mivel mi nem szimulátort csinálunk, így ez a céljainknak tökéletesen megfelel.

Akárcsak a beépített kerékhez, ehhez is tartoznak debug vonalak, és az irányítás/kerék helyének lekérdezése is annak megfelelően működik.

Persze ez sem egy tökéletes kerékszimuláció, például ha a kocsi egy helyben áll, akkor látható rajta egy kis rezgés. Ez azért van, mert amikor már a kocsi nagyon közel van a megálláshoz (valójában általában nem áll meg), akkor már csak egy nagyon pici erő kell ahhoz, hogy ténylegesen megállítsa a kocsit. Ehhez az lenne szükséges, hogy a kerekek tudjanak arról, hogy a többi kerék éppen mekkora erőt fejt ki, mely sajnos nem része a rendszernek, így ez egy lehetséges irány a komponens továbbfejlesztésére ² (bár szóljon mentségemre, ez a probléma a Unity-s kerék esetében is létezik).



Többjátékos mód

A játék a gépi tanulós ellenfelek mellett lehetőséget nyújt arra is, hogy a barátainkkal is játsszunk. Mivel sima UDP alapú³ kommunikációra akarunk építkezni (és szerverért nem akarunk fizetni), így egy helyi hálózati többjátékos módra esett a választás.

A többjátékos mód megvalósításához a Unity beépített Netcode csomagját használtam, mely lehetőséget ad arra, hogy a játékos hostoljon, vagy csatlakozzon egy más által hostolt játékhoz.

² Ha a kerekek tudnak egymásról, az magában nem teszi könnyűvé a problémát. Ahhoz, hogy tényleg állóhelyzetbe hozzuk a kocsit, nem elég a lendületét kinullázni, hanem a perdületével is foglalkozni kell. Ehhez szükség van a kocsi tetszőleges tengely körüli tehetetlenségi nyomatékának kiszámolására, amelyet a Unity beépített merevtest komponenséből kinyerni egy elég számításigényes feladat.

³ Egy másik lehetőség lett volna pl. egy p2p megoldás, mely nem feltétlen igényelne szervert. A szerver elkerülése végett még egy UDP hole punching alapú megoldásra is gondoltunk, de sajnos ennek elég korlátozott a felhasználása, mivel a legtöbb routert nem olyan könnyű „kicselezni”

A használt technológia kiválasztása után a következő lépés annak megtervezése volt, hogy hogyan lesz szimulálva a játék. A következő két lehetőség jött szóba:

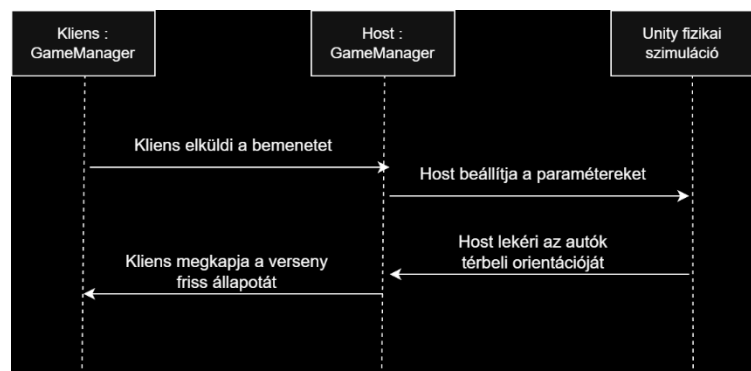
- *Szerveroldali szimuláció:* A kliens csak játékos inputot küld a szervernek és a szervertől kapott pozíció-információk alapján jeleníti meg a verseny állapotát
- *Helyi szimuláció:* Mind a szerver, mind a kliens futtat fizikai szimulációt. A szimulált testeket leíró adatok küldődnek el mindkét féltől; a szerver a kapott adatokat validálja, a kliens pedig azok alapján korrigálja az esetleges eltéréseket

Az autók nagy sebessége és komplex felépítése miatt a szerveroldali szimulációra esett a választás (ezzel a megoldással a csalás is egy kicsit nehezebb, bár még így is megvan a módja).

Az egymáshoz való csatlakozást a Unity Netcode beépített NetworkManager szkriptje kezeli, viszont a játék futását egy, a hálózaton is singleton GameManager (innen a Menedzser) objektum irányítja. A Menedzser küldi el az újonnan érkező klienseknek a, már a lobby-ban levő játékosok adatait. Szintén a Menedzser szinkronizálja a játék stádiumait (rajt, játékmenet, célbaérkezések követése).

Míg a játék stádiumainak kezelése triviális, érdemes egy picit alaposabban belenézni, hogyan is zajlik le az autó irányítása.

A kliens az éppen aktuális állapot alapján (lehetőleg minden fizikai frame-nél) elküldi a bemenetet a hostnak (pl. nagy gázzal jobb kanyar). A host az általa szimulált világban az adott klienshez tartozó autó paramétereit beállítja a kliens által küldött értékekre. Bár a mellékelt szekvencia diagram egy kicsit csalóka, a host igazából a kliens által küldött adatok megérkezésétől függetlenül minden fizikai frame-kor az összes kliensnek elküldi a verseny aktuális állapotát.



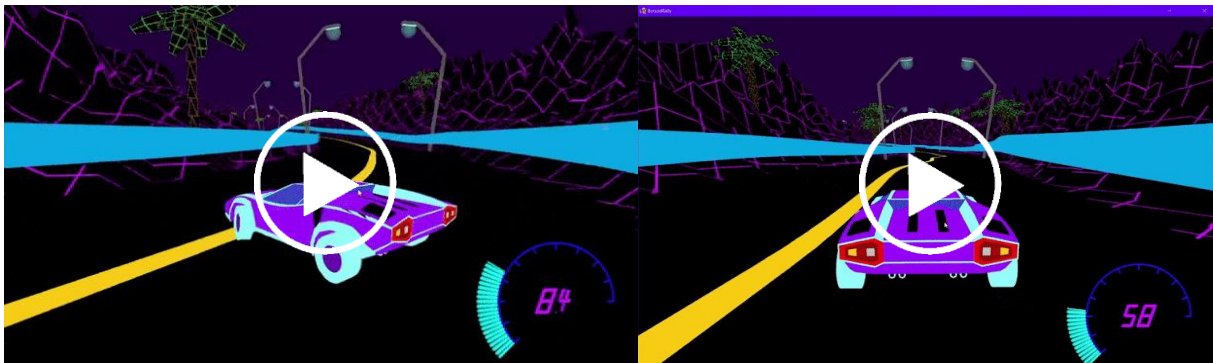
Ez a megoldás a fizikai szimulációt leegyszerűsíti, viszont a kliens által látott játékállapotot nagyon kiteszi a hálózati viszonyoknak (a kliens játéka legjobb esetben is 50 frissítés per másodperccel fut, ráadásul az input is késik legalább $RTT + 0,02$ másodpercet).

Ennek a problémának a kiküszöböléséhez a kliensnek a kapott állapotokat valahogy „ki kell simítania”. A feladat jellemzően kétféleképpen van megoldva:

- *Kliens oldali interpoláció:* A kliens a már megkapott állapotok között interpolál. Ez a megoldás sima játékmenetet biztosít azzal a hátránnyal, hogy a kliens mindig késni fog a host-hoz képest

- *Kliens oldali extrapoláció:* A kliens (nagyjából) szerveridőben frissíti az állapotát, a korábbi állapotok alapján. Elméletben egy tökéletes extrapoláció előre lát minden esemény, így teljes szinkronban van a szerverrel. A gyakorlatban viszont ez nem igazán lehetséges, és amennyiben kimarad egy-két csomag, az extrapoláció nagyon félremehet.

Az általunk választott megoldás az interpoláció, mivel egy helyi hálózaton számíthatunk arra, hogy nincsen nagy késleltetés a kliens és a host között (az általunk beállított érték egy 50 ms-os késleltetés). Az implementációm a következő eredményeket hozza (a baloldali az interpoláció nélküli eset⁴, a jobboldali pedig a működő interpoláció).



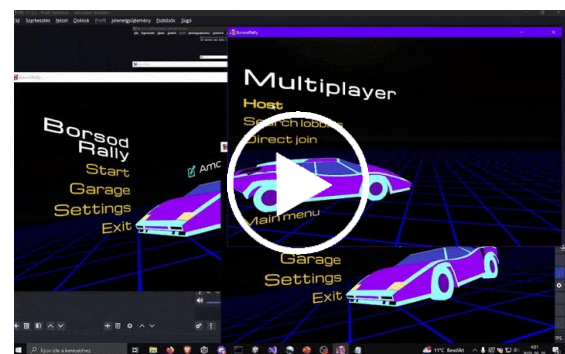
Ezekkel a többjátékos mód már majdnem készen van, egy dolog azonban még hiányzik. Hogyan tud a játékos csatlakozni egy már meglévő lobby-hoz?

A játék erre két lehetőséget ad:

- Hálózat felderítése
- Közvetlen csatlakozás

A hálózat felderítése a következőképpen működik:

1. A host GameManager-e egy külön szálon várja az üzeneteket.
2. A kliens MenuManager-e lobby keresésénél csinál egy új szálát, mely a host-tal való kommunikálásra fog szolgálni
3. A kliens küld egy UDP broadcast üzenetet a kiválasztott hálózati interfészhez⁵ tartozó hálózatra



A hálózat felderítésének menete

⁴ Az játék „darabossága” érezhető normális körülmények között is, viszont ahhoz, hogy az akadozás a mellékelt alacsony frissítési rátájú GIF-en is látszódjon, a Unity Netcode beépített hálózati viszony szimulátorában egy 5%-os csomag kimaradást állítottam be.

⁵ A játékos kiválaszthatja, hogy melyik hálózati interfészen host-ol / melyik hálózati interfésszel keres

4. Ha a hálózaton van egy aktív lobby, akkor a host elküldi a lobbyhoz tartozó adatokat (host neve, host üzenetfogadó címe szabad helyek száma, összes helyek száma)
5. A kliens az adatok megkapása után a lobby listában megjeleníti a lobby-hoz tartozó adatokat, egy „Join” gombbal együtt
6. A „Join” gomb megnyomásával a kliens (immáron nem broadcasttal) küld egy üzenetet a host-nak, hogy csatlakozhat-e. Amennyiben igen, a host visszaküldi a hozzátartozó NetworkManager címét, a kliens pedig csatlakozik rá.

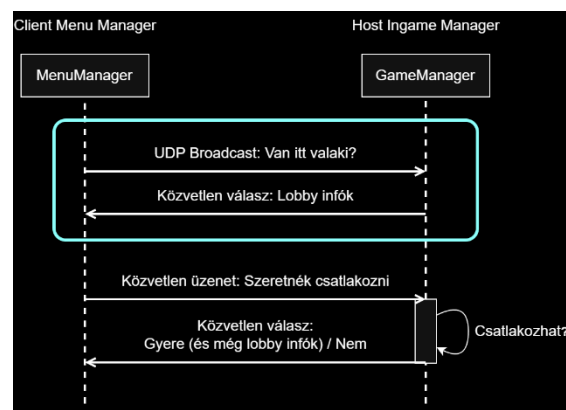
Amennyiben a hálózat felderítése sikertelen (pl. a router nem továbbít broadcast üzeneteket), a játékosnak megvan a lehetősége direkt csatlakozni. A képernyőn megjelenő InputField⁶ annak a lobby-nak az IP-címét várja, amelyhez a játékos csatlakozni szeretne. Ebben az esetben a lobby címét a host-oló játékostól kell megkérdeznie (a host-nál látszik az IP-címe a várakozó képernyőn). A csatlakozás lefolyása az előzőhöz képest annyiban változik, hogy a 3-as és a 4-es pont kimarad.



A közvetlen csatlakozás bemutatása (az elérhető hálózatok itt a 100.84.81.128/29 és a 192.168.56.0/24 voltak)

A csatlakozás menetét az jobboldalt látható diagram illusztrálja (a türkizzel bekeretezett részek csak a felderítés módban zajlanak le).

Most, hogy a játékos végre tud is csatlakozni a lobby-khoz, végre hátradőlhetünk, mert a többjátékos mód készen van. Hogy zárásként Madách Imrét idézzem; „Be van fejezve a nagy mű, igen. A gép forog, az alkotó pihen. Évmilliókig eljár tengelyén, Mig egy kerékfogát ujítani kell.”



⁶ Ez az InputField leellenőrzi, hogy a játékos helyes IP-címet adott e meg, sőt még azt is, hogy a cím valóban olyan hálózathoz tartozik-e, amely a játékos gépe által elérhető

Kutatásomhoz felhasznált források

- [EA Codemasters előadás](#)
- [Code Monkey videója az MLAgents-ről](#)
- [Megerősítéses tanulás elmélet](#)
- [Trackmania AI Reddit poszt](#)
- [Outrun AI kutatás a WPI hallgatói által](#)
- [Unity MLAgents dokumentáció](#)
- [Unity WheelCollider hangolás](#)
- [Toyful Games videója a kerékszimulációról](#)
- [Code Monkey videója a Netcode-ról](#)
- [Unity Netcode dokumentáció](#)
- [C# UDPSocket dokumentáció \(itt különösen sok időt töltöttem el\)](#)